

# LangGraph: Persistence and Human-in-the-Loop

LangGraph supports workflows that need to **persist state, pause execution, wait for human input, and resume later**. This is useful for order approvals, compliance checks, exception handling, and other business workflows.

## 1. Persistence with Checkpointers

A normal graph executes from start to finish:

```
START → Create Order → Process Order → END
```

Real workflows may need to stop:

```
START → Create Order → Approval → [WAIT] → Process → END
```

A **checkpoint** saves graph state at different points during execution.

```
graph = builder.compile(  
    checkpoint=checkpointer  
)
```

A **thread\_id** identifies a particular workflow execution:

```
config = {  
    "configurable": {  
        "thread_id": "ORDER-1001"  
    }  
}
```

This allows LangGraph to maintain independent state for different orders.

```
ORDER-1001  
├── checkpoint  
├── checkpoint  
└── checkpoint  
  
ORDER-1002  
├── checkpoint  
└── checkpoint
```

---

## 2. Persistent Storage

An in-memory checkpointer is useful for simple examples, but its state disappears when the process ends.

Persistent checkpointers such as SQLite allow state to survive application restarts.

```
Memory
  ↓
Process stops
  ↓
State lost
```

Compared with:

```
SQLite
  ↓
Process stops
  ↓
State remains
  ↓
Workflow can continue
```

---

## 3. Human-in-the-Loop

LangGraph can pause a workflow using `interrupt()`.

For example:

```
decision = interrupt({
    "message": "Human approval required",
    "order_id": state["order_id"],
    "amount": state["amount"]
})
```

Execution stops at this point.

```
Create Order
  ↓
Request Approval
  ↓
interrupt()
  X
  |
```

```

    | Graph paused
    |
    ↓
Human interaction

```

The important point is that `interrupt()` **does not provide the user interface**.

The surrounding application must present the request to a human.

For example, a simple application might use:

```
answer = input("Approve? (yes/no): ")
```

A real application could instead use a web UI, REST API, or administrative interface.

## 4. Resuming After Human Approval

After the human makes a decision, the application resumes the graph:

```
graph.invoke(
    Command(resume=True),
    config
)
```

The value passed to `resume` becomes the result of the interrupted operation.

For example:

```

Human: YES
  ↓
Command(resume=True)
  ↓
interrupt() returns True
  ↓
Process Order

```

For rejection:

```
Command(resume=False)
```

Thus, the graph can handle both approval and rejection.

## 5. State Inspection

The current workflow state can be retrieved using:

```
state = graph.get_state(config)
```

For example:

```
{  
  "order_id": "ORD1001",  
  "amount": 75000,  
  "status": "PENDING_APPROVAL",  
  "approved": False  
}
```

This is useful for monitoring workflow executions.

---

## 6. State History

A persistent graph maintains checkpoint history.

```
history = graph.get_state_history(config)
```

Conceptually:

```
Checkpoint 1  
  ↓  
Checkpoint 2  
  ↓  
Checkpoint 3  
  ↓  
Checkpoint 4
```

Each checkpoint represents a previous version of the workflow state.

---

## 7. Time Travel

LangGraph can use a historical checkpoint as the starting point for another execution path.

Suppose the original workflow was:

```
Amount = ₹75,000
  ↓
Approval
  ↓
Processed
```

We can select an earlier checkpoint:

```
Amount = ₹75,000
  ↓
Pending Approval
```

Modify the state:

```
Amount = ₹5,000
```

and resume execution from there.

This creates an alternate execution path rather than simply changing the final result.

---

## 8. Editing Historical State

A historical checkpoint can be modified using `update_state()`:

```
edited_config = graph.update_state(
    historical_checkpoint.config,
    {"amount": 5000}
)
```

The graph can then be invoked using the edited configuration.

This is useful for debugging, correcting workflow state, and exploring alternate execution paths.

When selecting a checkpoint, it is important to understand which node executed next and which state fields are available.

---

## 9. Important Consideration: Side Effects

An interrupted node can be executed again when the graph resumes.

Therefore, operations with external side effects need care.

Examples include:

```

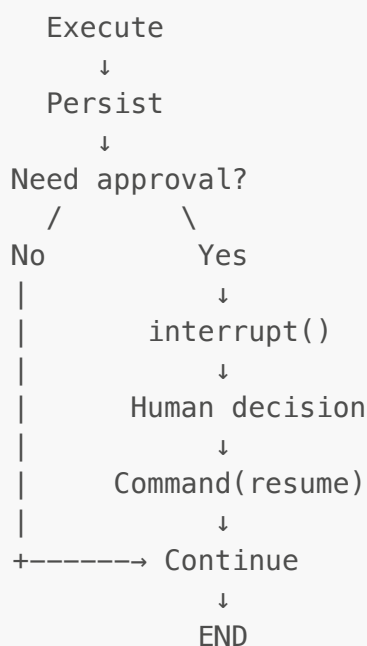
Charge credit card
Send email
Create shipment
Update external database

```

Such operations should be designed to be **idempotent** where appropriate, so resuming a workflow does not accidentally perform the operation twice.

## 10. Overall Flow

The complete concept can be summarized as:



### Key Takeaways

- **Checkpoints** persist graph state.
- **Thread IDs** identify workflow executions.
- **interrupt()** pauses execution.
- The **application provides the human interface**.
- **Command(resume=...)** continues execution with the human decision.
- **State history** provides access to previous checkpoints.
- **update\_state()** allows an alternate path to be created from historical state.
- External side effects should be handled carefully around interrupts and resumes.